INFT6201 — Data Science Project

Who Gets Hurt, Where and Why?

Severity analytics on New South Wales road crash data, 2020–2024

Quang Thanh Dong
Student ID: [your student number]

Master of Information Technology, Trimester 3, 2026

Code and reproducible outputs: https://github.com/QuangThanhDS/nsw-road-crash-analytics

October 18, 2026

Executive summary

New South Wales (NSW) recorded 340 road deaths in 2024, and the State's 2026 Road Safety Action Plan commits to halving deaths and cutting serious injuries by 30% by 2030 (Transport for NSW, 2022, 2025a). This report uses the Transport for NSW open crash data for 2020–2024 (Transport for NSW, 2025b), which covers 92,082 police-reported crashes and 1,507 deaths, to ask which crash circumstances turn a crash into a fatal or serious one, and where those crashes happen.

Three findings stand out from the preliminary analysis:

- **Regional rural roads carry a disproportionate share of trauma.** They account for 12.7% of crashes but 39.2% of deaths. At the same posted speed limit, a crash outside Greater Sydney–Newcastle–Wollongong is roughly two to four times more likely to be fatal (5.0% vs 1.4% at 100 km/h).
- **Who is involved matters more than when or in what weather.** Crashes involving a motorcycle, pedestrian or cyclist, or a collision with a tree or pole, carry the highest odds of a killed or seriously injured (KSI) outcome. Weather and day of week add almost nothing.
- **Data quality can mislead a model.** The *Street lighting* field is left "unknown" far more often for minor crashes than for fatal ones, which leaks the outcome into the model. Removing it lowers the apparent ROC-AUC from 0.81 to a more honest 0.75.

A temporally validated gradient-boosting model ranks 2024 crashes well (the top risk decile has a 61% KSI rate against 5.5% in the bottom decile), and an interpretable logistic regression performs almost as well. That makes logistic regression the better fit for public-sector stakeholders who need to explain their decisions.

Contents

Executive summary **1**

1 Case study **3**

1.1 Context . . . 3

1.2 Application examples . . . 3

1.3 Data science questions . . . 3

1.4 Stakeholders . . . 3

2 Datasets **4**

2.1 Background . . . 4

2.2 Data quality issues . . . 5

3 Data modelling **5**

3.1 Conceptual data model . . . 5

3.2 Analytical problem formulation . . . 6

3.3 Pre-processing 8
3.4 Models 8
3.5 Validation design: how a good model is identified 9
3.6 Suitability for the case study 11

4 Data visualisation and communication of results **12**

5 Preliminary results **12**
5.1 Data characterisation and distributions 12
5.2 Initial modelling results 16
5.3 Interpretation and next steps 19

References **20**

A How the models work **22**
A.1 What goes in and what comes out 22
A.2 Logistic regression, step by step 22
A.3 Gradient boosting, step by step 24
A.4 Comparing the two models 26
A.5 Complete list of logistic regression inputs 26

B Code used to build the models **29**
B.1 Model-building code, explained 29
B.2 Complete source files 31

1 Case study

1.1 Context

Road trauma is a leading cause of preventable death worldwide (World Health Organization, 2023). In NSW, more than two-thirds of road deaths occur on country roads, even though country residents make up about one-third of the population (Transport for NSW, 2022). The 2026 Road Safety Action Plan sets targets of a 50% reduction in deaths and a 30% reduction in serious injuries from the 2018–2020 baseline by 2030 (Transport for NSW, 2022). Meeting these targets needs evidence on *which* crashes become severe, so that limited infrastructure, enforcement and education budgets can go where they prevent the most trauma.

1.2 Application examples

The analysis is designed to support decisions such as:

- **Safer-roads investment:** prioritising route segments with clusters of severe run-off-road and head-on crashes for barriers, rumble strips or tree and pole treatments.
- **Vulnerable road user programs:** targeting motorcycle, pedestrian and cyclist safety where these users make up a large share of KSI crashes.
- **Enforcement and patrol scheduling:** aligning police resources with the times and places where crashes are most likely to be severe, not just most frequent.
- **Speed-zone review:** using severity differences across speed limits and regions to inform reviews of 80–110 km/h rural zones.

1.3 Data science questions

Q1. Severity factors: Which road, environment and road-user characteristics are associated with a reported crash being fatal or serious (KSI), and how well can they predict it?

Q2. Regional gap: Why are regional crashes more severe than metropolitan ones, and does the gap persist after accounting for speed limit and crash type?

Q3. Hotspots: Where are crashes and severe crashes concentrated, and which local government areas (LGAs) carry the largest burden?

Q4. Temporal patterns: How do volume and severity vary by year (including the COVID-19 period), day of week and time of day?

This preliminary report answers Q1 with initial models and gives descriptive evidence for Q2–Q4.

1.4 Stakeholders

Table 1 lists the main stakeholders and how each would use the results.

Table 1: Key stakeholders and their interest in the analysis.

Stakeholder	Interest and use of results
Transport for NSW, Centre for Road Safety	Data custodian and policy owner; program targeting, progress against 2030 targets
Local councils	Local road treatments, 40 km/h zones, LGA-level benchmarking
NSW Police Force	Timing and location of enforcement (e.g. weekend mornings, regional highways)
NSW Health and emergency services	Regional trauma response times and hospital demand
Insurers (icare, CTP insurers)	Injury burden and claims risk
Road user advocacy groups	Evidence for motorcycle, cycling and pedestrian safety
General public	Transparent communication of risk

2 Datasets

2.1 Background

The primary source is the *NSW Road Crash Data 2020–2024* release published by Transport for NSW on its Open Data Hub under a Creative Commons Attribution licence (Transport for NSW, 2025b). The release is updated annually as a rolling five-year window; the 2020–2024 release (updated 9 October 2025) is the most recent available. It contains crashes that meet national reporting criteria: the crash was reported to police, occurred on a public road, involved at least one moving vehicle, and resulted in a death, an injury or a vehicle being towed away (Transport for NSW, 2021).

The data comes as two linked tables (Table 2). The *Data Manual* defines every field (Transport for NSW, 2021). Severity levels below "fatal" come from linking police reports with health records: a *serious injury* crash is one where at least one person was matched to a hospital admission on the day of the crash or the day after, while a *moderate injury* is matched only to an emergency department attendance. Fatalities are deaths within 30 days of the crash.

To map the data, simplified Australian Statistical Geography Standard (ASGS) Edition 3 boundaries for the state, Greater Sydney and 2024 LGAs were used (Australian Bureau of Statistics, 2021). They were obtained in converted form from the `nswgeo` package (Suster, 2025).

Table 2: Datasets used in the analysis.

Table	Size	Content
CRASH	92,189 rows × 50 columns	One row per crash: year, month, weekday, two-hour time band, location (coordinates, LGA, suburb, street), road and environment (speed limit, alignment, surface, weather, lighting), crash type (RUM/DCA codes, first impact type), key vehicle type, and counts of people killed and injured by severity.
TRAFFIC UNIT	170,962 rows × 10 columns	One row per traffic unit (vehicle, pedestrian, cyclist): unit type, role in first impact, direction, manoeuvre and objects hit. Linked to CRASH by `Crash ID` (one-to-many).
ASGS boundaries	1 state, 1 GCCSA, 129 LGAs	Polygon boundaries (GDA2020) used for mapping.

2.2 Data quality issues

A systematic profile of all 50 CRASH fields (missing values, "unknown"-coded values, cardinality and range checks) and a referential-integrity check between tables found the issues summarised in Table 3. The structure of the data is clean: there are no duplicate crash IDs and no orphan traffic units, and every crash has at least one unit. The problems lie in *what was recorded and how*.

Table 3: Data quality issues identified and how they were handled.

Issue	Evidence	Handling
Outcome-dependent recording (target leakage)	*Street lighting* is "Unknown / not stated" for 36.5% of crashes overall, but for 62.8% of tow-away and 52.9% of minor-injury crashes compared with 0.6% of fatal ones. The KSI rate is 3.6% when the field is unknown and 26–38% otherwise.	Excluded from the main model; a comparison run quantifies the leakage (Section 5).
Structural missingness	*Primary temporary / hazardous / permanent feature* are 98.5%, 98.0% and 73.0% blank; *Other TU type* is 30.4% blank (single-vehicle crashes). A blank means "none", not "unknown".	Not used yet; to be recoded as "None".
Coverage and under-reporting	Only police-reported injury or tow-away crashes are included; minor crashes without a tow-away are absent.	Results describe severity *given* a reported crash, not crash risk per trip.
Late reporting	107 crashes from 2019 appear in reporting year 2020, and 63–167 crashes each year are reported in the following year.	Analysis uses *Year of crash* and keeps 2020–2024 only (92,082 crashes).
Consistency with official totals	Deaths in the data match the official 2023 toll (340) exactly, but 2024 shows 327 against an official 340 (Transport for NSW, 2025a).	Noted; 2024 counts may be revised in later releases.
Definitional overlap	*Urbanisation* is itself derived from LGA and speed limit (country urban $\leq$80 km/h, non-urban >80 km/h; Transport for NSW, 2021).	Regions are compared as metro vs rest of NSW *within* speed limits.
Coarse time and privacy limits	Time is given only as two-hour bands, month and weekday. No driver age, licence, speeding, alcohol or restraint fields are released.	Time treated as categorical; human factors listed as a limitation.
Spatial outliers	Five crashes at 159°E look anomalous but are on Lord Howe Island (part of NSW).	Kept; excluded only from the mainland map.

The first issue is the most consequential. Police record crash circumstances in more detail when someone is badly hurt. As a result, whether a field was *filled in* carries information about the outcome, which is a classic form of leakage (Kaufman et al., 2012). Smaller "unknown" categories in *Natural lighting* and *Key TU type* (each under 2.5% of crashes) show the same pattern and are flagged for treatment in the next stage.

3 Data modelling

3.1 Conceptual data model

The two source tables form a one-to-many relational model (Figure 1). For analysis, the TRAFFIC UNIT table was aggregated to the crash level as binary flags (for example *any pedestrian involved*,

any motorcycle involved, *tree or pole hit*) and joined to CRASH. This gives one row per crash, which matches the unit of the severity outcome.

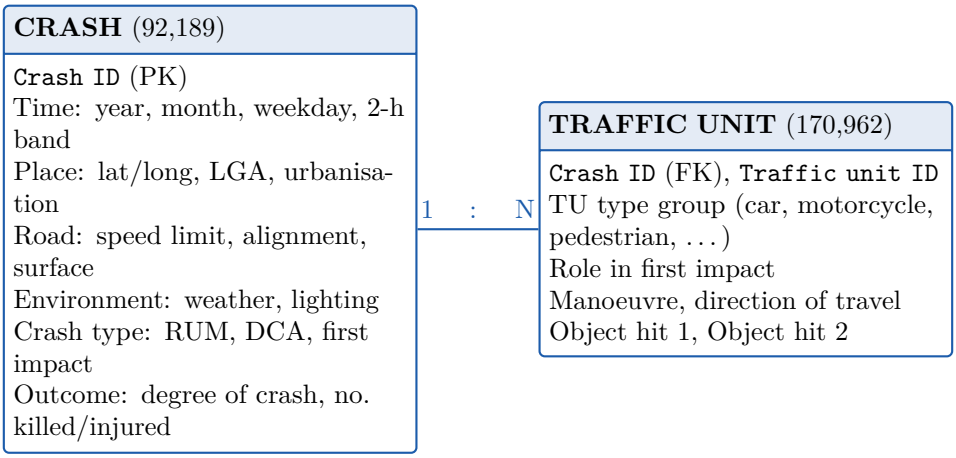


Figure 1: Relational model of the TfNSW crash data. Traffic units are aggregated to crash-level flags before modelling.

3.2 Analytical problem formulation

Unit of analysis. Each observation is one police-reported crash (92,082 crashes from 2020 to 2024). The model asks: *given that a crash has happened and been reported, how likely is it that someone was killed or seriously injured?* It does not predict whether a crash will happen, because the data contains no exposure measure such as traffic volume or kilometres travelled.

What is predicted (target). Crash severity is an ordered, five-level outcome, but its extremes are very unbalanced (1.5% fatal against 32.0% non-casualty). The preliminary model therefore predicts a binary target that matches the policy target in the Road Safety Action Plan (Transport for NSW, 2022):

$$y_i = \mathrm{KSI}_i = \begin{cases} 1 & \text{if } \textit{Degree of crash – detailed} \in \{\text{Fatal, Serious Injury}\} \\ 0 & \text{if Moderate Injury, Minor/Other Injury or Non-casualty (tow-away)} \end{cases} \tag{1}$$

Overall, 22.8% of crashes are KSI (22.2% in the training years and 25.1% in the 2024 test year). The model outputs a probability $\hat{p}_i = P(y_i = 1 \mid \mathbf{x}_i)$, which is turned into a "high-risk / not high-risk" label with a threshold (Section 3.5). The target is computed in `clean_crash()` (Listing 5, line 23).

Predictors. Predictors were restricted to circumstances that are known *at the scene*, before anyone's injuries are assessed. This gives 23 predictors in total (Table 4): 16 categorical attributes taken directly from the CRASH table, and 7 crash-level variables derived from the TRAFFIC UNIT table by aggregating its 170,962 unit records to one row per crash (Listing 5, lines 33–46). For example, `has_motorcycle` equals 1 if *any* unit in the crash was a motorcycle.

Table 4: Predictors used in the models. "Levels" is the number of categories (categorical) or the share of crashes with the flag set to 1 (binary).

Predictor	Type	Levels	Examples / rationale
Road and location (CRASH table)			
Speed limit	Categorical	12	50, 60, 100 km/h; impact speed drives injury energy
Urbanisation	Categorical	6	Sydney metro, country urban, country non-urban
Road classification (admin)	Categorical	3	State, regional, local road
Type of location	Categorical	13	2-way undivided, T-junction, roundabout
Alignment	Categorical	3	Straight, curved
Road surface	Categorical	3	Sealed, unsealed
School zone active	Categorical	3	Not a school zone, inactive, active
Signals operation	Categorical	4	Signals on, off, none
Other traffic control	Categorical	17	Give way, stop sign, pedestrian crossing
Environment and time (CRASH table)			
Weather	Categorical	7	Fine, raining, fog
Surface condition	Categorical	4	Dry, wet, snow/ice
Natural lighting	Categorical	5	Daylight, darkness, dusk, dawn
Two-hour intervals	Categorical	13	16:00–17:59; captures traffic conditions
Day of week	Categorical	7	Weekday against weekend travel
Crash mechanism and vehicles (CRASH table)			
First impact type	Categorical	11	Head-on, rear end, vehicle–object, rollover
Key TU type	Categorical	36	Car, SUV, motorcycle, semi-trailer (key unit in the crash)
Derived from the TRAFFIC UNIT table			
No. of traffic units involved	Numeric	1–26	Single- against multi-vehicle crashes (mean 1.85)
has_pedestrian	Binary	5.4%	Unprotected road user involved
has_motorcycle	Binary	11.7%	Unprotected rider involved
has_pedal_cycle	Binary	4.1%	Cyclist involved
has_heavy_vehicle	Binary	7.6%	Heavy rigid truck, articulated truck or bus; mass disparity
hit_tree_or_pole	Binary	10.2%	Rigid roadside object struck
has_parked_vehicle	Binary	9.7%	A parked (uncontrolled) vehicle involved

Variables deliberately excluded. Several fields were left out because they would give the model information it could not have in practice, or because they add noise without adding signal (Table 5).

Table 5: Fields excluded from the predictors and the reason.

Field(s)	Reason for exclusion
No. killed, seriously, moderately and minor injured; Degree of crash	These define the target. Including them would let the model read the answer (target leakage; Kaufman et al., 2012).
Street lighting	Completeness depends on the outcome (Section 2); it inflates ROC-AUC from 0.75 to 0.81 for the wrong reason.
Crash ID, street, town, route no., identifying feature, coordinates	Identifiers or very high-cardinality location text; spatial questions are handled separately (Q3).
Year, month, reporting year	Year defines the train/test split; month adds seasonality with little severity signal at this stage.
RUM and DCA codes (86+ levels)	Detailed crash-type codes that largely duplicate *First impact type*; kept for later work.
Primary permanent / temporary / hazardous feature	73–99% blank (structural missingness); to be recoded first.
LGA (130 levels), Conurbation	LGA is too granular for a first model; Conurbation duplicates Urbanisation.

3.3 Pre-processing

All pre-processing is wrapped inside scikit-learn `Pipeline` objects (Pedregosa et al., 2011), so every step is learned from the training years only and then applied unchanged to the test year.

1. **Cleaning** (Listing 5, lines 18–30): keep crash years 2020–2024; derive KSI; parse speed limits; treat coordinates outside NSW as missing.
2. **Aggregation and join**: compute the seven TRAFFIC UNIT flags per crash with a `groupby("Crash ID").max()` and left-join them to CRASH (Listing 5, lines 33–52).
3. **Missing values**: blank categorical values become an explicit `"Missing"` level. Existing "Unknown" codes are kept as their own level, so the model can still use them (Listing 6, line 50).
4. **Encoding**: for logistic regression, one-hot encoding with categories seen fewer than 50 times in training grouped into a single "infrequent" column. This produces 128 input columns. For gradient boosting, each category is mapped to an integer code and split natively as a category, with unseen categories mapped to -1 (Listing 6, lines 55–68).
5. **Class weighting**: to stop the majority class from dominating, each class k receives weight $w_k = n/(2\,n_k)$. With 22.2% KSI in training, a KSI crash counts about 2.25 times in the loss and a non-KSI crash about 0.64 times (He & Garcia, 2009).

3.4 Models

Three models were fitted. Their hyperparameters are summarised in Table 6.

(a) Majority-class baseline. This model ignores the predictors and assigns every crash the training KSI rate (22.2%). It defines the floor any useful model must clear: ROC-AUC = 0.5, and PR-AUC equal to the prevalence.

(b) Logistic regression. This is the standard model for binary crash-severity outcomes (Savolainen et al., 2011). It models the log-odds of KSI as a linear function of the encoded predictors:

$$\log \frac{\hat{p}_i}{1-\hat{p}_i} = \beta_0 + \sum_{j=1}^{128} \beta_j x_{ij}, \qquad \hat{p}_i = \frac{1}{1+e^{-(\beta_0+\boldsymbol{\beta}^\top \mathbf{x}_i)}} \tag{2}$$

The coefficients are estimated by minimising the class-weighted log-loss plus an L2 (ridge) penalty, which shrinks unstable coefficients for rare categories towards zero:

$$\min_{\boldsymbol{\beta}} \; -\sum_{i=1}^{n} w_{y_i} \left[y_i \log \hat{p}_i + (1-y_i) \log(1-\hat{p}_i) \right] + \frac{1}{2C} \, \|\boldsymbol{\beta}\|_2^2, \qquad C = 0.5 \tag{3}$$

Each coefficient converts to an **odds ratio** e^{β_j}: the multiplicative change in the odds of KSI when that predictor is present, holding the others fixed. An odds ratio of 5.7 for `has_motorcycle` means the odds of a KSI outcome are 5.7 times higher when a motorcycle is involved. Because every category level is encoded and the penalty shrinks coefficients, odds ratios are read as relative effects rather than comparisons against a single reference level. The solver (L-BFGS) converged in 92 iterations.

(c) Histogram gradient boosting. Gradient boosting builds an additive ensemble of shallow regression trees, where each new tree h_m is fitted to the gradient of the log-loss left by the trees before it (Friedman, 2001):

$$F_M(\mathbf{x}) = F_0 + \eta \sum_{m=1}^{M} h_m(\mathbf{x}), \qquad \hat{p} = \frac{1}{1+e^{-F_M(\mathbf{x})}} \tag{4}$$

Here $\eta = 0.05$ is the learning rate and each tree has at most 31 leaves. The "histogram" variant bins continuous values into at most 255 bins before searching for splits, as in LightGBM (Ke et al., 2017), and splits categorical predictors natively by grouping categories. Because each tree splits on several predictors in sequence, the model can learn interactions (for example, a motorcycle crash on a 100 km/h curve) and non-linear effects without them being specified. Regularisation comes from the small learning rate, the leaf limit, an L2 penalty on leaf values, and **early stopping**. scikit-learn holds out 10% of the *training* rows and stops adding trees when the validation loss has not improved for 10 rounds. Training stopped after 105 of a maximum of 400 trees, which guards against overfitting.

Table 6: Model specifications and hyperparameters. Values were fixed in advance and not tuned on the test year.

	Logistic regression	**Histogram gradient boosting**
scikit-learn class	`LogisticRegression`	`HistGradientBoostingClassifier`
Input encoding	One-hot, `min_frequency=50` (128 columns)	Ordinal codes, native categorical splits
Loss	Class-weighted log-loss	Class-weighted log-loss
Regularisation	L2, $C = 0.5$	`l2_regularization=1.0`, `max_leaf_nodes=31`
Learning	L-BFGS, `max_iter=3000` (converged at 92)	$\eta = 0.05$, up to 400 trees (early-stopped at 105)
Class imbalance	`class_weight="balanced"`	`class_weight="balanced"`
Randomness	Deterministic	`random_state=42`
Interpretation	Odds ratios e^{β_j}	Permutation importance

3.5 Validation design: how a good model is identified

Temporal hold-out. The intended use is to learn from past years and flag high-risk crash profiles in a new year. The validation therefore mimics that use: models are trained on 2020–2023 (73,143 crashes) and evaluated once on 2024 (18,939 crashes), a year they have never seen (Figure 2). An out-of-time test is stricter than a random split because it exposes the model to year-to-year drift. In 2024 the KSI rate rose to 25.1%, against 22.2% in training. It also prevents crashes from the same period, with the same reporting practices, from appearing on both sides of the split (Tashman, 2000).

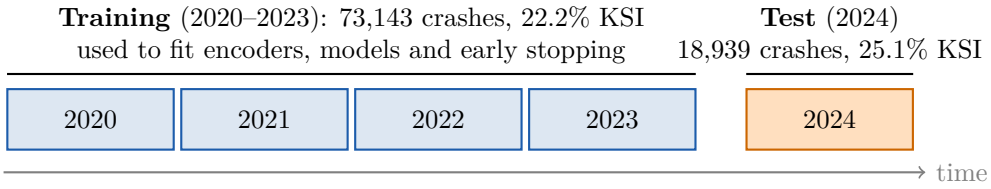


Figure 2: Temporal validation design. All fitting (encoders, coefficients, trees, early stopping) uses 2020–2023 only; 2024 is scored once.

Controls against optimistic results.

- *No information from the test year* is used for fitting. Encoders and models sit in one pipeline fitted on the training rows, and gradient boosting's early-stopping set is carved out of the training years.
- *Hyperparameters were fixed in advance* (Table 6) and not tuned against 2024, so the test score is an honest estimate. Tuning will use the training years only (see below).

- *Leaky and target-defining fields are excluded* (Table 5). A comparison run that includes *Street lighting* is reported separately to show the size of the leakage effect.

Evaluation metrics. Because only about a quarter of crashes are KSI, accuracy would be misleading: the baseline that never predicts KSI is 75% accurate in 2024 but useless. The metrics below are used instead (Fawcett, 2006; He & Garcia, 2009; Saito & Rehmsmeier, 2015). Let TP, FP, TN and FN be the true/false positives and negatives when a crash is labelled high-risk if $\hat{p}_i \geq 0.5$.

- **ROC-AUC** (discrimination): the probability that a randomly chosen KSI crash receives a higher predicted risk than a randomly chosen non-KSI crash. 0.5 is random, 1.0 is perfect. It does not depend on the threshold.
- **PR-AUC** (average precision): the area under the precision–recall curve. Its random baseline equals the KSI prevalence (0.251 in 2024), so it is more sensitive than ROC-AUC to performance on the minority class (Saito & Rehmsmeier, 2015).
- **Recall (sensitivity)** $= \mathrm{TP}/(\mathrm{TP} + \mathrm{FN})$: the share of actual KSI crashes the model flags. For road safety, missing a severe crash is the costlier error, so recall is prioritised.
- **Precision** $= \mathrm{TP}/(\mathrm{TP} + \mathrm{FP})$: the share of flagged crashes that really were KSI, which indicates how many resources would go to false alarms.
- **Fatal crashes flagged**: recall measured on fatal crashes only, the outcome stakeholders care most about.
- **Calibration by risk decile**: crashes are sorted into ten equal groups by $\hat{p}$, and the mean predicted risk is compared with the observed KSI rate in each group. A well-ranked model shows observed rates rising steadily across deciles. A well-calibrated one has predicted and observed rates that match (Niculescu-Mizil & Caruana, 2005).

The 0.5 threshold is applied to the class-weighted model's output, which places it roughly at the training prevalence. In practice the threshold can be moved to trade recall against precision, for example to fit an agency's inspection budget.

Criteria for a "good" model. A model is judged suitable for this case study if it meets the criteria in Table 7. Together they cover performance, generalisation, stability and interpretability. Common rules of thumb treat a ROC-AUC of 0.7–0.8 as acceptable and 0.8–0.9 as excellent discrimination (Hosmer et al., 2013).

Table 7: Criteria used to identify a good model, with the evidence for the two candidate models (2024 test year unless stated).

Criterion	Test	Logistic regression	Gradient boosting
Better than chance	ROC-AUC > 0.5; PR-AUC $>$ prevalence (0.251)	0.742; 0.483 ✓	0.749; 0.498 ✓
Acceptable discrimination	ROC-AUC ≥ 0.7 (Hosmer et al., 2013)	✓	✓
Catches severe crashes	High recall, especially on fatal crashes	65% KSI; 85% fatal	67% KSI; 85% fatal
No overfitting	Small gap between train and test ROC-AUC	0.742 vs 0.742	0.764 vs 0.749
Stable over time	Leave-one-year-out ROC-AUC within 2020–2023	0.730–0.748	0.734–0.754
Ranks risk usefully	Observed KSI rises across risk deciles	–	5.5% $\rightarrow$ 60.7%
Calibrated probabilities	Predicted $\approx$ observed per decile	Not yet (class weighting)	Not yet (top decile 0.79 vs 0.61)
Interpretable	Effects explainable to stakeholders	Odds ratios	Permutation importance only

Stability check. To check that the 2024 result is not a one-off, each model was also re-fitted four times on the training years, each time holding out one year (2020, 2021, 2022 or 2023) and scoring it (Listing 6, lines 100–108). ROC-AUC stayed between 0.730 and 0.754 across held-out years for both models. The lowest score was for 2021, the COVID-affected year, and the 2024 test score lies inside this range, so performance is consistent over time. The next stage will use this same leave-one-year-out scheme on the training years to tune hyperparameters and the decision threshold, keeping 2024 untouched.

How feature influence is measured. For logistic regression, influence is read from the odds ratios (Equation 2). For gradient boosting, which has no coefficients, **permutation importance** is used (Breiman, 2001): each predictor's values are randomly shuffled in the 2024 data, five times, and the average drop in ROC-AUC is recorded. A large drop means the model relies on that predictor (Listing 6, line 146).

3.6 Suitability for the case study

- **Logistic regression** is the most suitable *primary* model for this case study. The stakeholders are public agencies that must justify their investment decisions, and odds ratios translate directly into program priorities. It performs only 0.007 AUC below gradient boosting (Section 5).
- **Gradient boosting** works best as a *check* on the logistic model. Its small gain suggests that severity is driven mostly by additive main effects in this feature set. Its importance ranking independently confirms the logistic model's drivers.
- **Limitations of both:** predicted probabilities are inflated by class weighting and need recalibration before they are reported as risks (Niculescu-Mizil & Caruana, 2005). Neither model accounts for the unobserved heterogeneity that the accident-research literature stresses. Ordered and random-parameters models are the natural extension for the full five-level severity scale (Mannering & Bhat, 2014; Savolainen et al., 2011).
- **Spatial models** (density-based clustering of KSI crashes) are better suited than classifiers to Q3 and are planned for the next stage.

4 Data visualisation and communication of results

Each visual was chosen to answer one question for one audience, and all were produced with Matplotlib (Hunter, 2007) using a single consistent design system. Table 8 maps each visual to its question and audience.

Table 8: Visualisations, the questions they answer and their intended audience.

Visual (technique)	Q	Purpose	Audience
Severity bar chart (Fig. 3)	–	Class balance; justifies the KSI target	Analysts
Yearly trend lines (Fig. 4)	Q4	COVID dip and post-2022 rise	TfNSW, public
Day × time heatmaps (Fig. 5)	Q4	Volume against severity by time	Police
Speed limit × region lines (Fig. 6)	Q2	Regional gap at equal speed	TfNSW, councils
Impact-type 100% stacked bars (Fig. 7)	Q1	Severity share by crash mechanism	All
Hexbin maps (Fig. 8)	Q3	Where crashes and severe crashes are	Councils, TfNSW
ROC/PR curves (Fig. 9)	Q1	Model performance	Analysts
Odds ratios and importance (Figs. 10, 11)	Q1	Drivers of severity	Policy makers

Key design choices:

- **Rates alongside counts.** Volume and severity often point in different directions. The heatmaps (Fig. 5) and maps (Fig. 8) pair a count view with a KSI-share view, so that "where crashes are common" is not mistaken for "where crashes are dangerous".
- **Hexagonal binning for 92,000 points.** Plotting each crash as a point would create heavy overplotting along the coast. Hexbins aggregate points into equal-area cells (Carr et al., 1987). A logarithmic colour scale keeps sparse regional cells visible next to Sydney cells with thousands of crashes. Cells with fewer than 15 crashes are hidden on the KSI-share map to avoid unstable rates.
- **Geographic context.** ABS state, Greater Sydney and LGA boundaries (Australian Bureau of Statistics, 2021), plus labelled markers for Sydney, Newcastle and Wollongong, let non-technical readers orient themselves.
- **Accessible colour.** Single-hue sequential scales encode magnitude. Categorical series use a palette checked for colour-vision-deficiency separation, and series are also labelled directly or with a legend, so colour is never the only cue.
- **Small-sample suppression and annotation.** Groups with fewer than 100 crashes are hidden in the speed chart, and bar charts are annotated with sample sizes.
- **Models explained in policy terms.** Odds ratios are shown on a log scale centred at 1, so effects that raise and lower risk appear symmetrically.

5 Preliminary results

5.1 Data characterisation and distributions

Severity. Of the 92,082 crashes from 2020 to 2024, 32.0% were non-casualty tow-aways, 28.4% moderate injury, 16.8% minor injury, 21.3% serious injury and 1.5% fatal (1,388 crashes, 1,507

deaths; Figure 3). The KSI target therefore has a prevalence of 22.8%. Most crashes involve two traffic units (59%), and 29% are single-vehicle crashes.

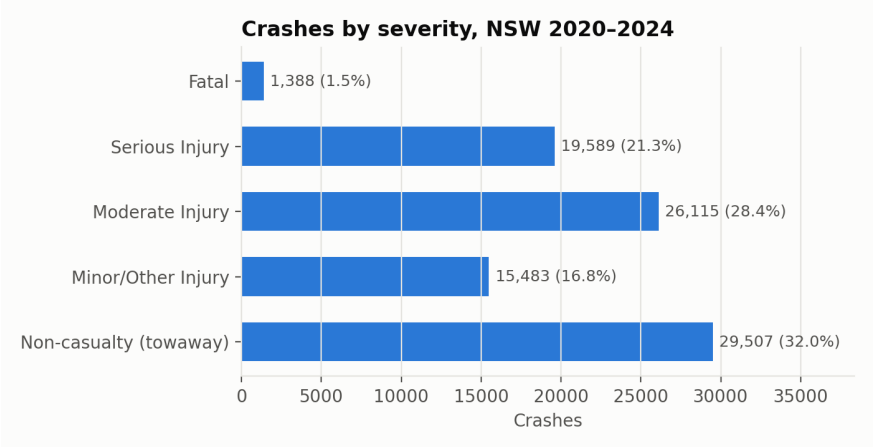


Figure 3: Distribution of crashes by severity, NSW 2020–2024.

Trend. Fatal crashes held at around 260 a year in 2020–2022 and rose to around 300 in 2023–2024. Serious-injury crashes fell in 2021, during COVID-19 lockdowns, and then rose 37% to 4,459 in 2024, while moderate, minor and tow-away crashes stayed flat (Figure 4). Severe crashes are therefore increasing, not crashes overall, which moves NSW away from its 2030 targets.

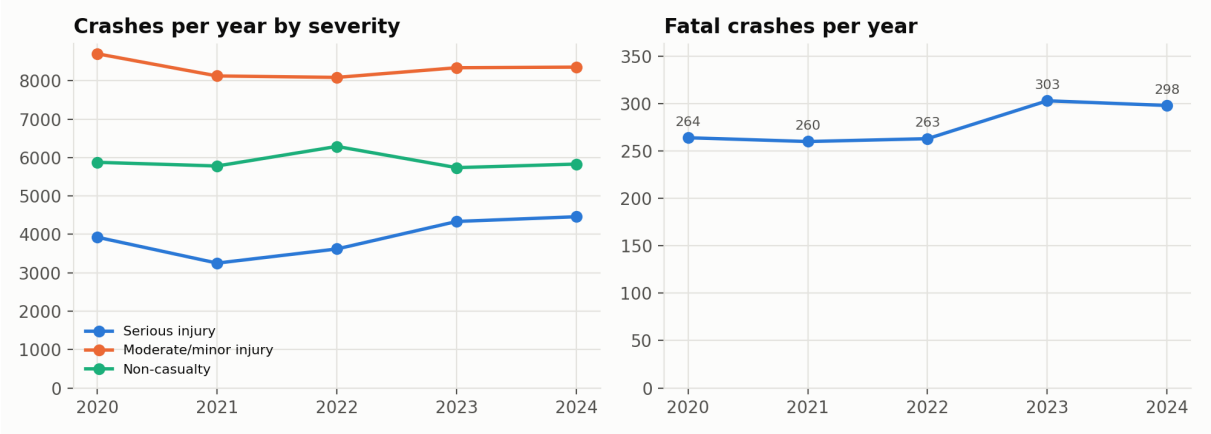


Figure 4: Crashes per year by severity (left) and fatal crashes per year (right).

Time of day and week. Crash volume peaks on weekday afternoons; Friday 16:00–17:59 is the busiest slot, with 2,304 crashes. Severity follows a different pattern: the highest KSI shares occur on *weekend mornings*, reaching 31% on Sunday 10:00–11:59 and 29% on Sunday 08:00–09:59, against about 20% in the weekday peak (Figure 5). Congested peak traffic produces many low-speed crashes, while lighter weekend traffic, often on recreational and regional routes, produces fewer but more severe ones.

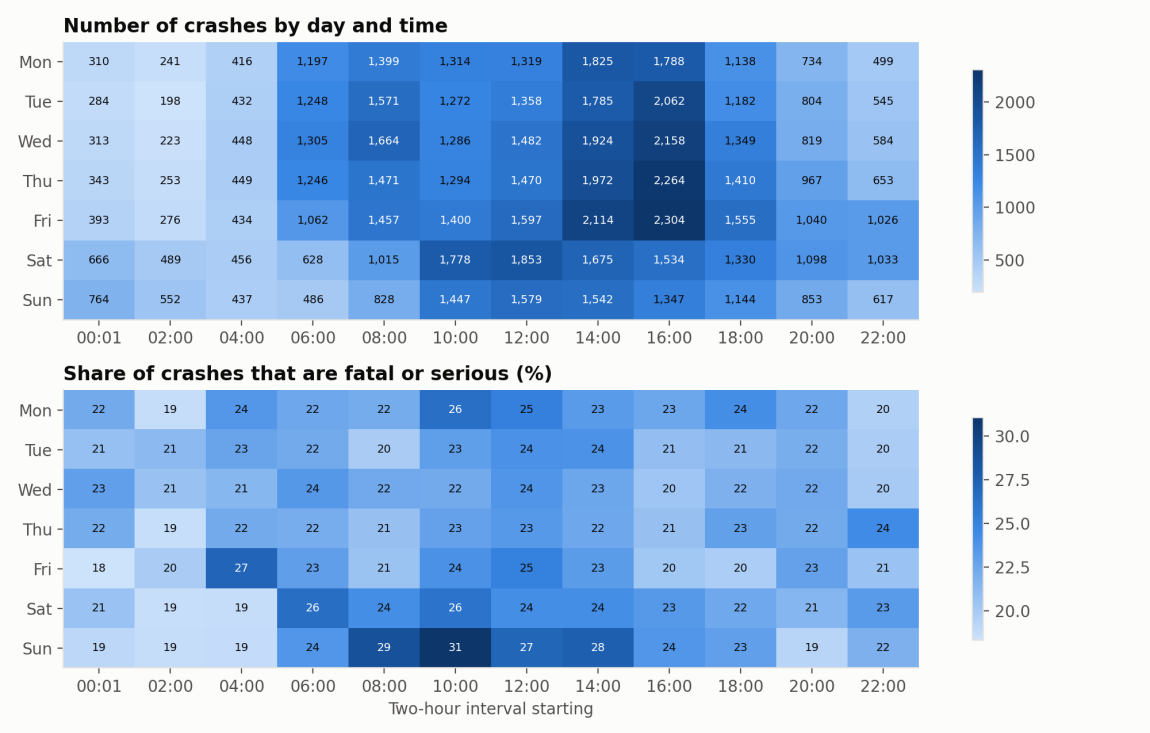


Figure 5: Crash counts (top) and share of crashes that are fatal or serious (bottom) by day and two-hour interval.

Regional gap (Q2). Table 9 shows that regional rural NSW has 12.7% of crashes but 39.2% of deaths, with a fatal-crash rate five times that of the metropolitan conurbation. This matches TfNSW's statement that more than two-thirds of deaths occur on country roads (Transport for NSW, 2022). Even at the same speed limit, crashes outside the conurbation are more severe (Figure 6): at 100 km/h, 5.0% are fatal against 1.4% in the metro areas, and at 80 km/h, 3.4% against 1.6%. Crash mechanism contributes to the gap: 74.8% of rural crashes are single-vehicle crashes (metro: 18.2%), and 45.1% are vehicle–object impacts (metro: 14.1%).

Table 9: Crashes, deaths and severity by region, 2020–2024.

Region	Crashes	Share of crashes	Deaths	Share of deaths	Fatal rate
Greater Sydney / Newcastle / Wollongong	62,551	67.9%	580	38.5%	0.9%
Regional urban	17,821	19.4%	337	22.4%	1.7%
Regional rural	11,690	12.7%	590	39.2%	4.5%

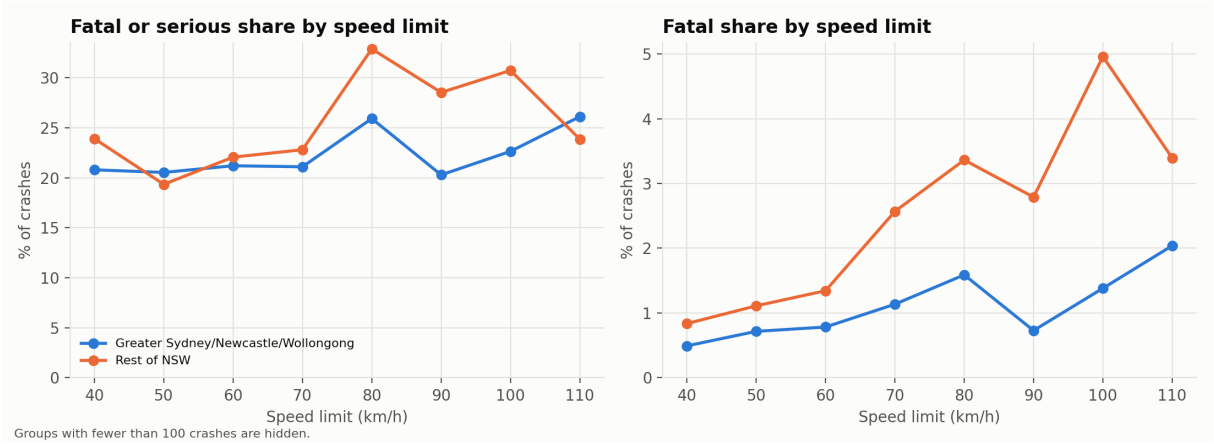


Figure 6: KSI share (left) and fatal share (right) by speed limit, metropolitan conurbation against the rest of NSW.

Crash mechanism. Pedestrian (47% KSI), person–object (46%), head-on (39%) and rollover (32%) crashes are the most severe, and rear-end crashes the least (13%; Figure 7). Head-on crashes have the highest fatal share, at 6.4%.

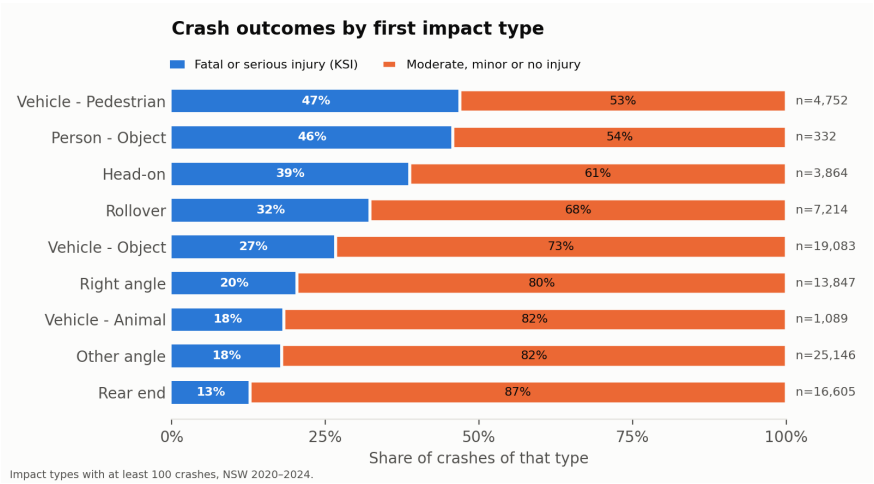


Figure 7: Crash outcomes by first impact type: share of crashes that were fatal or serious (blue) against moderate, minor or no injury (orange). Impact types with at least 100 crashes.

Spatial distribution (Q3). Crashes follow the population and the highway network, with the densest cells in Sydney and along the Pacific, Hume and New England corridors (Figure 8, left). Within Greater Sydney, KSI shares appear higher on the urban fringe and along major arterials than in the dense inner suburbs (Figure 8, right). By count, the LGAs with the most KSI crashes are Central Coast (1,068), Canterbury-Bankstown (990) and Blacktown (823).

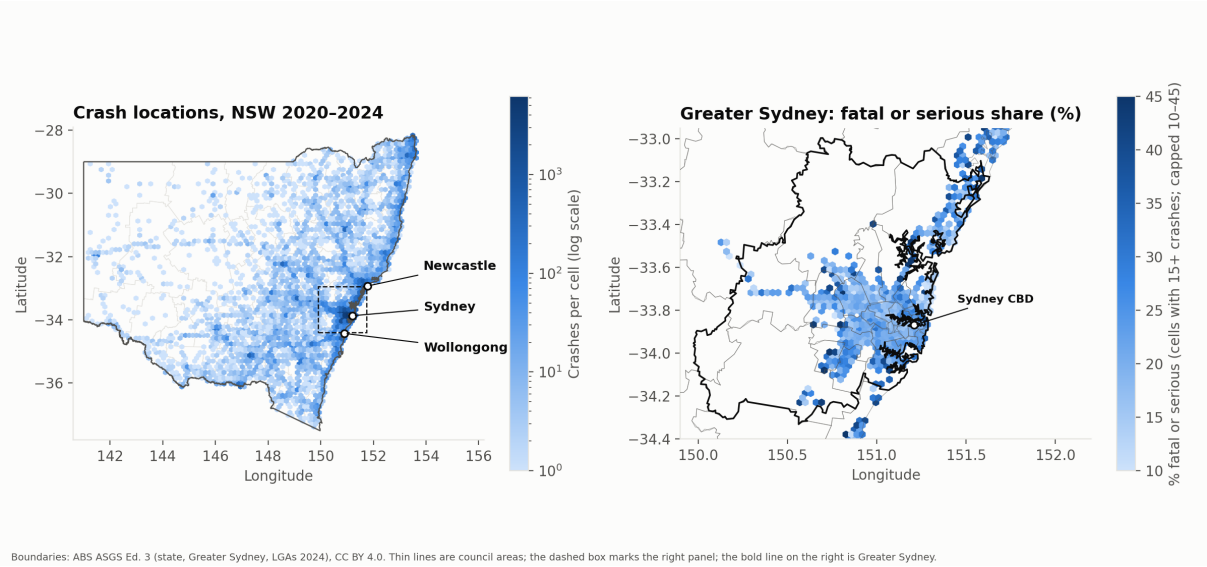


Figure 8: Crash density across NSW, log scale (left), and KSI share within Greater Sydney (right). Boundaries: ABS ASGS Edition 3.

5.2 Initial modelling results

Table 10 reports test-set performance on 2024 crashes. Both models clearly outperform the baseline. Gradient boosting achieves a ROC-AUC of 0.749 and a PR-AUC of 0.498, double the 0.251 that random ranking would give. Logistic regression is within 0.007 AUC. At the default threshold, the models identify about two-thirds of KSI crashes and 85% of fatal crashes. The ROC and PR curves almost coincide (Figure 9).

Table 10: Model performance on the 2024 test set (18,939 crashes, 25.1% KSI).

Model	ROC-AUC	PR-AUC	KSI recall	KSI precision	Fatal crashes flagged
Majority-class baseline	0.500	0.251	0.00	–	0%
Logistic regression	0.742	0.483	0.65	0.44	85%
Gradient boosting	**0.749**	**0.498**	0.67	0.43	85%
Gradient boosting incl. Street lighting (leaky)	*0.809*	*0.553*	*0.81*	*0.45*	*95%*

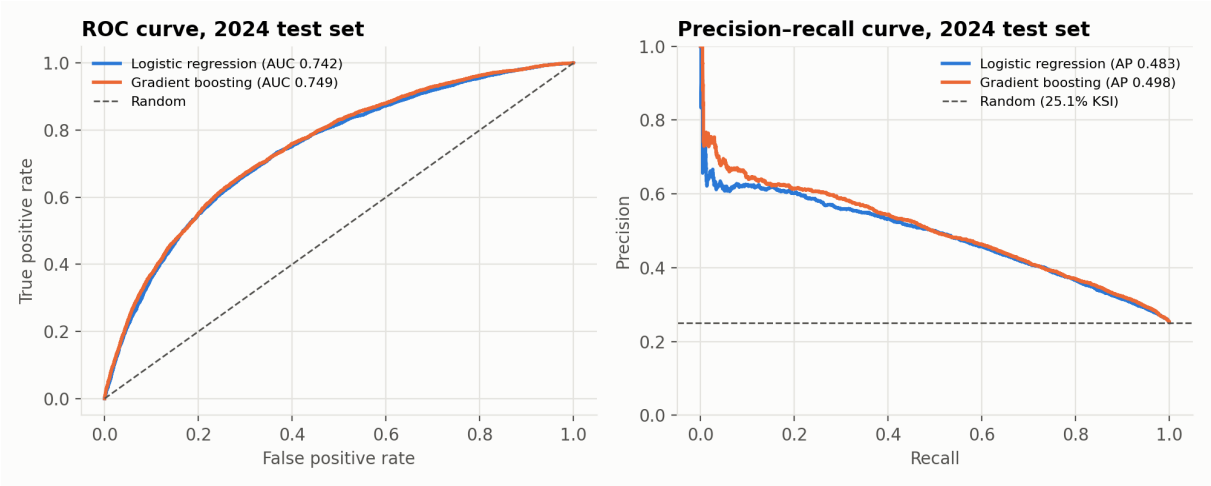


Figure 9: ROC (left) and precision–recall (right) curves on the 2024 test set.

Risk ranking. When 2024 crashes are sorted into deciles by predicted risk, the actual KSI rate rises steadily from 5.5% in the lowest decile to 60.7% in the highest, an eleven-fold spread. However, the top decile's mean predicted probability is 0.79, which confirms the over-estimation expected from class weighting and the need for recalibration.

Leakage check. Adding *Street lighting* raises ROC-AUC to 0.809 and fatal-crash recall to 95% (Table 10, last row). In that run, *Street lighting* becomes by far the most important feature, which only makes sense as a recording artefact. This demonstrates how the data-quality issue in Section 2 would inflate apparent performance, and why the leaky feature was removed.

Drivers of severity (Q1). The two models agree on the main drivers (Figures 10 and 11):

- **Motorcycle involvement** is the strongest factor (odds ratio ≈5.7; largest permutation importance). 49% of motorcycle crashes are KSI, compared with 19% of other crashes.
- **Vulnerable road users**: pedestrian (≈4.4), cyclist (≈4.3) and person–object (≈4.4) involvement, and motorised mobility scooters (≈7.2, though based on few cases).
- **Roadside objects and crash mechanism**: hitting a tree or pole (≈2.5), head-on impact (≈2.0) and heavy-vehicle involvement (≈1.8).
- **Speed environment**: 100–110 km/h zones (≈1.5–1.7). Rear-end crashes (≈0.46), animal strikes (≈0.42) and 40 km/h zones (≈0.53) are associated with lower severity.
- Weather, day of week and school-zone status contribute almost nothing once these factors are known.

Remaining "unknown" categories (for example *Natural lighting: Unknown*, odds ratio ≈0.17) show the same recording effect and will be handled in the next stage. These are associations and should not be read as causal effects.

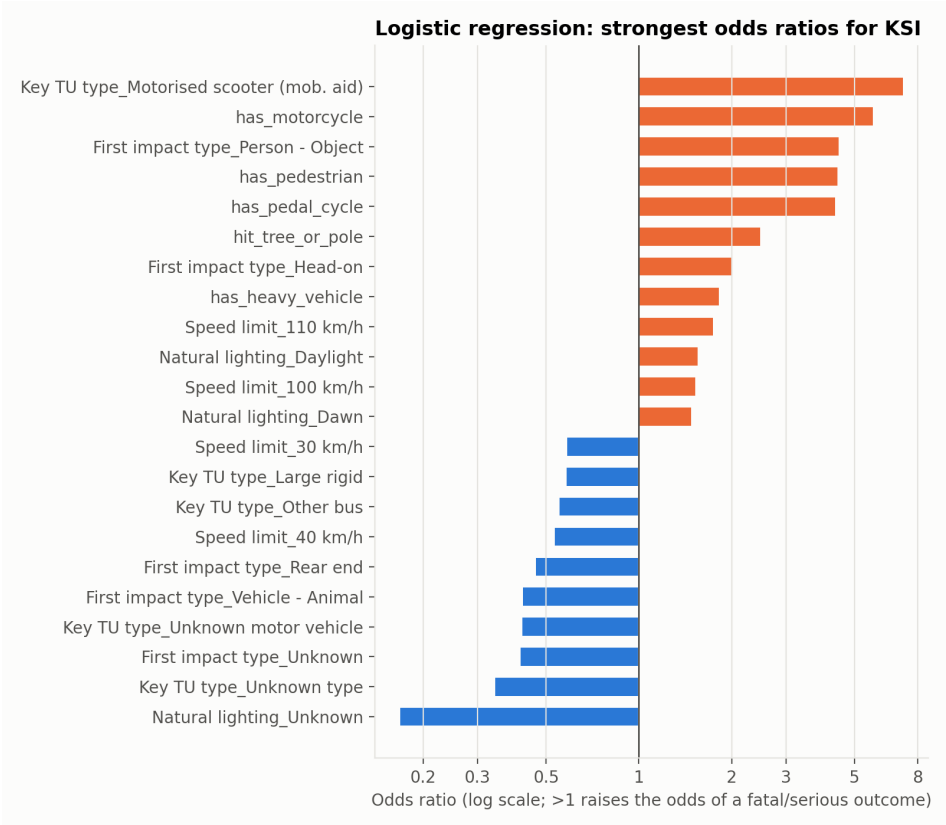


Figure 10: Strongest logistic-regression odds ratios for a KSI outcome (log scale).

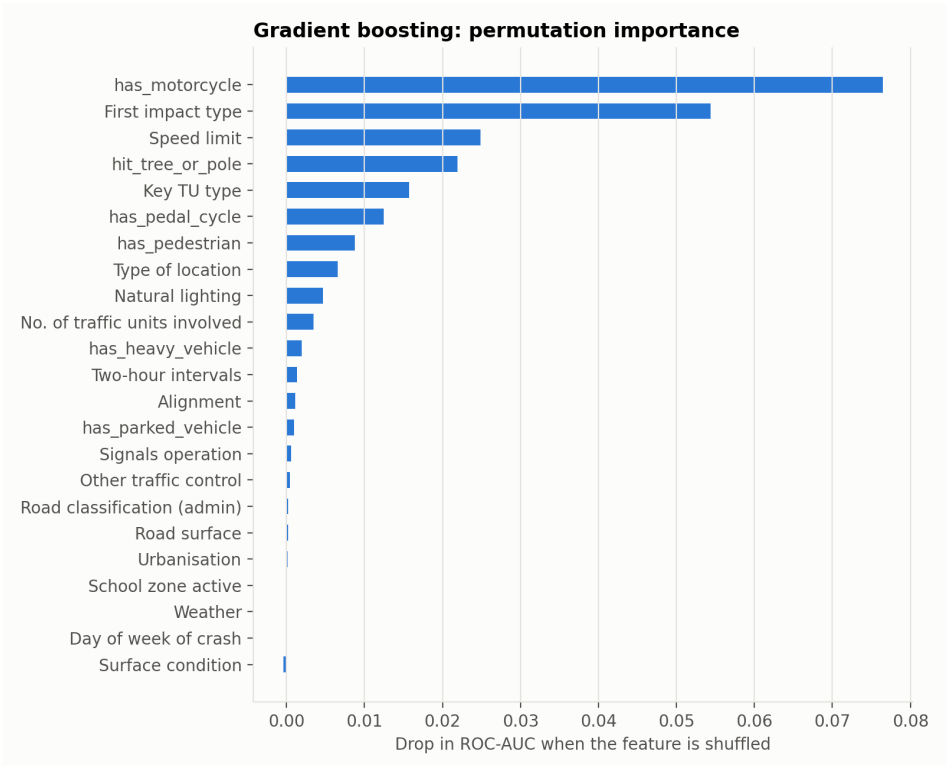


Figure 11: Gradient-boosting permutation importance (drop in test ROC-AUC when a feature is shuffled).

5.3 Interpretation and next steps

A ROC-AUC of about 0.75 is moderate, and the ceiling reflects the data rather than the model: the main determinants of injury severity, such as travel speed, restraint use, impairment, occupant age and vehicle safety rating, are not in the open release. Even so, the models separate low- and high-risk crash profiles clearly and give consistent, policy-relevant drivers. The next stage will:

1. recode structural blanks, treat the remaining "unknown" levels, and recalibrate probabilities;
2. add ABS population data to compare LGAs by per-capita KSI rate (Q3);
3. model the regional gap directly, controlling for speed, mechanism and vehicle mix (Q2);
4. apply density-based clustering to KSI crash locations to identify hotspots (Q3);
5. extend to the full ordered severity scale and a fatal-only model, and check stability with rolling-year validation.

References

Australian Bureau of Statistics. (2021). *Australian statistical geography standard (ASGS) Edition 3, July 2021–June 2026* [Licensed under CC BY 4.0]. ABS. https://www.abs.gov.au/statistics/standards/australian-statistical-geography-standard-asgs-edition-3/jul2021-jun2026

Breiman, L. (2001). Random forests. *Machine Learning*, *45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Carr, D. B., Littlefield, R. J., Nicholson, W. L., & Littlefield, J. S. (1987). Scatterplot matrix techniques for large N. *Journal of the American Statistical Association*, *82*(398), 424–436. https://doi.org/10.1080/01621459.1987.10478445

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, *27*(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, *29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, *21*(9), 1263–1284. https://doi.org/10.1109/TKDE.2008.239

Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3rd ed.). Wiley. https://doi.org/10.1002/9781118548387

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, *6*(4), 1–21. https://doi.org/10.1145/2382577.2382579

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, *30*, 3146–3154.

Mannering, F. L., & Bhat, C. R. (2014). Analytic methods in accident research: Methodological frontier and future directions. *Analytic Methods in Accident Research*, *1*, 1–22. https://doi.org/10.1016/j.amar.2013.09.001

Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. *Proceedings of the 22nd International Conference on Machine Learning*, 625–632. https://doi.org/10.1145/1102351.1102430

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, *10*(3), e0118432. https://doi.org/10.1371/journal.pone.0118432

Savolainen, P. T., Mannering, F. L., Lord, D., & Quddus, M. A. (2011). The statistical analysis of highway crash-injury severities: A review and assessment of methodological alternatives. *Accident Analysis & Prevention*, *43*(5), 1666–1676. https://doi.org/10.1016/j.aap.2011.03.025

Suster, C. (2025). *Nswgeo: Geospatial data and maps for New South Wales, Australia* (R package; Version 0.6.0.9000). https://github.com/cidm-ph/nswgeo

Tashman, L. J. (2000). Out-of-sample tests of forecasting accuracy: An analysis and review. *International Journal of Forecasting*, *16*(4), 437–450. https://doi.org/10.1016/S0169-2070(00)00065-0

Transport for NSW. (2021, September). *Definitions and notes to support road crash data*. Centre for Road Safety, Transport for NSW. https://opendata.transport.nsw.gov.au/dataset/nsw-crash-data

Transport for NSW. (2022). *2026 road safety action plan: Toward zero trauma on NSW roads*. Transport for NSW. https://towardszero.nsw.gov.au/sites/default/files/2022-04/TNSW9659%20Road-Safety-Action-Plan-2026-ACC_1.pdf

Transport for NSW. (2025a, January 2). *Safer driving is the New Year's resolution NSW needs to lower road toll* [Media release]. Transport for NSW. https://www.transport.nsw.gov.au/system/files/media/documents/2025/250102%20MR%20Graham%20Aitchison%20annual%20road%20toll%202024.pdf

Transport for NSW. (2025b, October 9). *NSW crash data: NSW road crash data 2020–2024 (CRASH and TRAFFIC UNIT tables)* (Version 2020–2024 release) [Licensed under CC BY]. https://opendata.transport.nsw.gov.au/dataset/nsw-crash-data

World Health Organization. (2023). *Global status report on road safety 2023*. World Health Organization. https://www.who.int/publications/i/item/9789240086517

A How the models work

This appendix explains, step by step, how the two predictive models in Section 3 turn the predictors of a crash into a risk of a killed-or-seriously-injured (KSI) outcome. It lists every predictor each model uses and works through two real crashes from the 2024 test year. The code that builds and runs the models is in Appendix B.

A.1 What goes in and what comes out

Both models receive the same 23 predictors for every crash (Table 4): 16 categorical descriptions of the road, environment, time and crash mechanism, plus 7 crash-level counts and flags derived from the TRAFFIC UNIT table. Both return one number per crash: an estimated probability $\hat{p}$ that the crash is KSI. A crash is labelled *high risk* if $\hat{p} \geq 0.5$.

The two models read the predictors differently:

- **Logistic regression** cannot read text categories. Each categorical predictor is therefore *one-hot encoded*: *Speed limit*, for example, becomes 12 columns (`Speed limit_50 km/h`, `Speed limit_60 km/h`, ...), and a crash has a 1 in the column for its speed limit and 0 in the others. Levels seen fewer than 50 times in training are pooled into one "infrequent" column per predictor. The 16 categorical predictors become 121 columns, and with the 7 numeric and binary predictors the model sees **128 input columns** (Table 16).
- **Gradient boosting** uses the 23 predictors directly. Categories are stored as integer codes, and the trees split them as groups of categories (for example {Head-on, Rollover} against the rest).

A.2 Logistic regression, step by step

Step 1 — add up a score. Every input column j has a learned weight β_j. For a given crash, the model starts from an intercept β_0 and adds the weight of every column that is switched on (multiplied by its value for the numeric predictor). The total is the crash's *log-odds score* z:

$$z = \beta_0 + \beta_{\text{speed=100}} + \beta_{\text{impact=Vehicle–Object}} + \cdots + \beta_{\text{units}} \times (\text{no. of units}) + \beta_{\text{motorcycle}} \times 1.$$

A positive weight pushes the crash towards KSI, and a negative weight pushes it away.

Step 2 — turn the score into a probability. The logistic (sigmoid) function squeezes any score into the range 0 to 1: $\hat{p} = 1/(1 + e^{-z})$. A score of 0 gives 0.5, large positive scores approach 1, and large negative scores approach 0.

Step 3 — learn the weights. The weights are chosen to make the training crashes' observed outcomes as likely as possible (maximum likelihood), which means minimising the log-loss in Equation 3. Two adjustments are made. First, each KSI crash counts about 2.25 times and each non-KSI crash about 0.64 times (class weighting), so the minority class is not ignored. Second, an L2 penalty ($C = 0.5$) pulls weights towards zero, which stops rare categories from receiving extreme weights. The optimiser (L-BFGS) adjusts all 129 numbers (128 weights plus the intercept) together until the loss stops improving; this took 92 iterations.

Step 4 — read the weights. Raising e to a weight gives an *odds ratio*. An odds ratio of 5.72 for `has_motorcycle` means that, other predictors being equal, the odds of KSI are 5.72 times higher when a motorcycle is involved. Because every level of a categorical predictor has its own column and the penalty shrinks them, the weights should be compared *between levels of the same predictor*. For example, *Rear end* (-0.77) against *Head-on* ($+0.69$) shows how much the impact

type changes the score. A level should not be read in isolation. Table 16 lists every weight and odds ratio the model learned.

Worked example. Table 11 shows two real 2024 crashes: a motorcycle that left a curved 100 km/h country road and struck an object (crash 1360953, actual outcome: serious injury), and a two-car rear-end crash on a 60 km/h divided road in metropolitan Sydney (crash 1343582, actual outcome: moderate injury). Each was chosen as the most typical member (median predicted risk) of its crash profile in 2024. Table 12 shows how logistic regression adds up their scores.

Table 11: Predictor values of the two worked-example crashes (2024 test year).

Predictor	**High-risk example (#1360953)**	**Low-risk example (#1343582)**
Urbanisation	Country non-urban	Sydney metro. area
Speed limit	100 km/h	60 km/h
Weather	Unknown	Fine
Natural lighting	Daylight	Daylight
Surface condition	Dry	Dry
Road surface	Sealed	Sealed
Alignment	Curved	Curved
Type of location	2-way undivided	Divided road
Road classification (admin)	Regional	State
First impact type	Vehicle - Object	Rear end
Key TU type	Motorcycle	Car (sedan/hatch)
Two-hour intervals	10:00 - 11:59	12:00 - 13:59
Day of week of crash	Wednesday	Sunday
School zone active	Not a school zone	Not a school zone
Signals operation	Nil	Nil
Other traffic control	No traffic controls	No traffic controls
No. of traffic units involved	1	2
has_pedestrian	0	0
has_motorcycle	1	0
has_pedal_cycle	0	0
has_heavy_vehicle	0	0
hit_tree_or_pole	0	0
has_parked_vehicle	0	0
Actual outcome	Serious Injury (KSI = 1)	Moderate Injury (KSI = 0)

Table 12: Logistic regression score for the two example crashes: each predictor's contribution $\beta_j x_j$ to the log-odds z, then the probability.

Contribution to z from...	**High-risk**	**Low-risk**
Intercept β_0	-0.448	-0.448
Urbanisation	-0.072	-0.016
Speed limit	+0.421	-0.069
Weather	-0.049	+0.032
Natural lighting	+0.440	+0.440
Surface condition	-0.031	-0.031
Road surface	-0.203	-0.203
Alignment	-0.141	-0.141
Type of location	+0.111	-0.112
Road classification (admin)	-0.107	-0.193
First impact type	+0.045	-0.768
Key TU type	+0.193	+0.083
Two-hour intervals	-0.023	-0.006
Day of week of crash	-0.081	-0.047
School zone active	-0.212	-0.212
Signals operation	+0.128	+0.128
Other traffic control	-0.073	-0.073
No. of traffic units involved	+0.248	+0.497
has_pedestrian	+0.000	+0.000
has_motorcycle	+1.743	+0.000
has_pedal_cycle	+0.000	+0.000
has_heavy_vehicle	+0.000	+0.000
hit_tree_or_pole	+0.000	+0.000
has_parked_vehicle	+0.000	+0.000
Log-odds score z	**+1.890**	**-1.139**
Probability $\hat{p} = 1/(1+e^{-z})$	**0.869**	**0.242**
Label at threshold 0.5	High risk	Not high risk

For the motorcycle crash, the single largest push comes from `has_motorcycle` (+1.74), helped by the 100 km/h limit (+0.42). The score of +1.89 gives a probability of 0.87, and the crash is correctly flagged as high risk. For the rear-end crash, the impact type pulls the score down strongly (-0.77), giving $z = -1.14$ and $\hat{p} = 0.24$, so it is correctly not flagged. Both probabilities are higher than the true risk because of class weighting (Section 3.5); the *ranking* of the two crashes is what matters.

A.3 Gradient boosting, step by step

Gradient boosting builds its prediction from many small decision trees, each correcting the mistakes of the ones before it (Friedman, 2001; Ke et al., 2017).

Step 1 — start from a neutral guess. Every crash starts with the same score F_0. With balanced class weights this corresponds to a probability of about 0.5.

Step 2 — measure the errors. For each training crash, the model computes how far its current probability is from the truth (the gradient of the log-loss: roughly $y_i - \hat{p}_i$). KSI crashes with low predicted risk have large positive errors, and non-KSI crashes with high predicted risk have large negative ones.

Step 3 — fit a small tree to the errors. A decision tree with at most 31 leaves is grown to separate crashes with similar errors. At each split, the tree tries every predictor. For numeric predictors it tries thresholds over at most 255 histogram bins; for categorical ones, groupings of categories. It keeps the split that best reduces the error. A tree might, for instance, first split on `has_motorcycle`, then on *First impact type* $\in$ {Head-on, Vehicle–Object, Rollover}, then on

Speed limit $\geq 90\,\text{km/h}$. Each leaf stores a correction value.

Step 4 — take a small step. Each crash's score is moved by its leaf's correction multiplied by the learning rate $\eta = 0.05$. Small steps mean no single tree dominates, which makes the model more robust.

Step 5 — repeat, and stop at the right time. Steps 2–4 are repeated, with each new tree focusing on what is still wrong. scikit-learn sets aside 10% of the training crashes and stops once 10 consecutive trees fail to improve the loss on them (early stopping). Training stopped after **105 trees**.

Step 6 — predict. A new crash is passed down all 105 trees; its leaf corrections are added to F_0, and the total score is turned into a probability with the same sigmoid function as logistic regression (Equation 4).

Because a path down a tree combines several predictors (motorcycle *and* 100 km/h *and* curved road), gradient boosting learns interactions automatically. The cost is that there is no single weight per predictor to read. Instead, **permutation importance** measures how much the 2024 ROC-AUC drops when one predictor's values are randomly shuffled (Table 14).

Worked example. Table 13 follows the same two crashes as the trees are added. Both start near 0.5. The motorcycle crash's risk climbs as successive trees recognise its high-risk profile, while the rear-end crash's risk falls. Most of the movement happens in the first 50 trees, which is why early stopping ended training at 105.

Table 13: Gradient boosting probability for the two example crashes after m trees.

Trees used (m)	**High-risk example**	**Low-risk example**
1	0.517	0.485
5	0.575	0.430
10	0.631	0.377
25	0.737	0.289
50	0.796	0.241
105	0.802	0.224
Label at threshold 0.5	High risk	Not high risk

Table 14: Gradient boosting permutation importance for all 23 predictors (mean drop in 2024 ROC-AUC over five shuffles). Values near zero mean the model barely relies on the predictor; a small negative value is random noise.

Rank	Predictor	Drop in ROC-AUC
1	has_motorcycle	0.0764
2	First impact type	0.0544
3	Speed limit	0.0248
4	hit_tree_or_pole	0.0220
5	Key TU type	0.0158
6	has_pedal_cycle	0.0125
7	has_pedestrian	0.0088
8	Type of location	0.0066
9	Natural lighting	0.0047
10	No. of traffic units involved	0.0035
11	has_heavy_vehicle	0.0019
12	Two-hour intervals	0.0014
13	Alignment	0.0012
14	has_parked_vehicle	0.0010
15	Signals operation	0.0006
16	Other traffic control	0.0004
17	Road classification (admin)	0.0002
18	Road surface	0.0002
19	Urbanisation	0.0001
20	School zone active	0.0001
21	Weather	0.0000
22	Day of week of crash	0.0000
23	Surface condition	-0.0004

A.4 Comparing the two models

Table 15: How the two models differ in practice.

	Logistic regression	Gradient boosting
Form of the model	One weighted sum of 128 inputs	Sum of 105 small trees
Predictor interactions	Only if added by hand	Learned automatically
Explaining a prediction	Exact: add up the contributions (Table 12)	Indirect: importance, staged predictions
2024 test ROC-AUC	0.742	0.749
Best use here	Primary model for stakeholders	Check on whether non-linear effects are missed

A.5 Complete list of logistic regression inputs

Table 16 lists all 128 encoded inputs, grouped by the original predictor, with the learned weight β_j and odds ratio e^{β_j}. Within each predictor, levels are sorted from the most to the least severe association.

Table 16: All logistic regression inputs with learned weights and odds ratios.

Predictor	Level / encoded column	β_j	Odds ratio
Urbanisation	Wollongong met. area	-0.007	0.99
	Sydney metro. area	-0.016	0.98
	Newcastle met. area	-0.018	0.98
	Country urban	-0.032	0.97
	Country non-urban	-0.072	0.93
	(infrequent levels, <50 rows)	-0.248	0.78

Table 16 continued

Predictor	Level / encoded column	β_j	Odds ratio
Speed limit	110 km/h	+0.555	1.74
	100 km/h	+0.421	1.52
	80 km/h	+0.345	1.41
	90 km/h	+0.217	1.24
	70 km/h	+0.112	1.12
	60 km/h	-0.069	0.93
	10 km/h	-0.185	0.83
	(infrequent levels, <50 rows)	-0.257	0.77
	50 km/h	-0.373	0.69
	30 km/h	-0.532	0.59
	40 km/h	-0.627	0.53
Weather	Raining	+0.038	1.04
	Fine	+0.032	1.03
	Fog or mist	-0.001	1.00
	Overcast	-0.015	0.98
	Unknown	-0.049	0.95
	Other	-0.150	0.86
	Snowing	-0.247	0.78
Natural lighting	Daylight	+0.440	1.55
	Dawn	+0.389	1.47
	Darkness	+0.328	1.39
	Dusk	+0.231	1.26
	Unknown	-1.781	0.17
Surface condition	Unknown	+0.300	1.35
	Dry	-0.031	0.97
	Wet	-0.176	0.84
	Snow or ice	-0.487	0.61
Road surface	Unknown	+0.031	1.03
	Sealed	-0.203	0.82
	Unsealed	-0.221	0.80
Alignment	Curved	-0.141	0.87
	Straight	-0.252	0.78
Type of location	Single limited access	+0.150	1.16
	Other	+0.141	1.15
	2-way undivided	+0.111	1.12
	Multiple intersection	+0.085	1.09
	T-junction	+0.073	1.08
	Y-junction	+0.018	1.02
	X-intersection	+0.010	1.01
	Dual freeway	+0.001	1.00
	(infrequent levels, <50 rows)	-0.091	0.91
	Divided road	-0.112	0.89
	L-junction	-0.201	0.82
	1-way street	-0.250	0.78
	Roundabout	-0.329	0.72
Road classification (admin)	Local	-0.093	0.91
	Regional	-0.107	0.90
	State	-0.193	0.82
First impact type	Person - Object	+1.492	4.44
	Head-on	+0.690	1.99
	Vehicle - Pedestrian	+0.244	1.28
	(infrequent levels, <50 rows)	+0.176	1.19
	Vehicle - Object	+0.045	1.05
	Rollover	+0.007	1.01
	Right angle	-0.166	0.85
	Other angle	-0.368	0.69
	Rear end	-0.768	0.46
	Vehicle - Animal	-0.863	0.42
	Unknown	-0.880	0.41
Key TU type	Motorised scooter (mob. aid)	+1.972	7.18
	Passenger van	+0.379	1.46
	(infrequent levels, <50 rows)	+0.361	1.43
	Motorised push scooters	+0.225	1.25
	Motorcycle	+0.193	1.21
	Pedal cycle	+0.190	1.21
	Light truck utility(from 2018)	+0.109	1.11
	Station wagon	+0.100	1.11
	Light truck	+0.086	1.09

Table 16 continued

Predictor	Level / encoded column	β_j	Odds ratio
	Car (sedan/hatch)	+0.083	1.09
	4 wheel drive / SUV	+0.046	1.05
	Motor scooter	+0.024	1.02
	Utility	-0.026	0.97
	STA bus	-0.219	0.80
	Semi-trailer	-0.413	0.66
	B-double (road train prior 98)	-0.435	0.65
	Large rigid	-0.540	0.58
	Other bus	-0.589	0.55
	Unknown motor vehicle	-0.867	0.42
	Unknown type	-1.070	0.34
Two-hour intervals	(infrequent levels, <50 rows)	+0.049	1.05
	20:00 - 21:59	+0.005	1.00
	22:00 - Midnight	+0.005	1.00
	18:00 - 19:59	-0.000	1.00
	12:00 - 13:59	-0.006	0.99
	00:01 - 01:59	-0.016	0.98
	04:00 - 05:59	-0.020	0.98
	10:00 - 11:59	-0.023	0.98
	06:00 - 07:59	-0.029	0.97
	14:00 - 15:59	-0.044	0.96
	02:00 - 03:59	-0.090	0.91
	08:00 - 09:59	-0.107	0.90
	16:00 - 17:59	-0.117	0.89
Day of week of crash	Monday	+0.008	1.01
	Sunday	-0.047	0.95
	Tuesday	-0.060	0.94
	Thursday	-0.061	0.94
	Friday	-0.071	0.93
	Wednesday	-0.081	0.92
	Saturday	-0.081	0.92
School zone active	Yes	+0.044	1.05
	Not a school zone	-0.212	0.81
	No	-0.226	0.80
Signals operation	On	+0.168	1.18
	Nil	+0.128	1.14
	Off	-0.242	0.79
	Unknown / not stated	-0.447	0.64
Other traffic control	No right turn	+0.155	1.17
	Other traf. control	+0.135	1.14
	Stop sign	+0.013	1.01
	Give way sign	-0.006	0.99
	(infrequent levels, <50 rows)	-0.028	0.97
	No traffic controls	-0.073	0.93
	Left turn only	-0.150	0.86
	Road/rail worker	-0.174	0.84
	Ped crossing	-0.264	0.77
No. of traffic units involved	(value)	+0.248	1.28
has_pedestrian	1 = yes	+1.483	4.41
has_motorcycle	1 = yes	+1.743	5.72
has_pedal_cycle	1 = yes	+1.466	4.33
has_heavy_vehicle	1 = yes	+0.597	1.82
hit_tree_or_pole	1 = yes	+0.905	2.47
has_parked_vehicle	1 = yes	-0.021	0.98

B Code used to build the models

Everything in this report was produced by three scripts in the project repository (https://github.com/QuangThanhDS/nsw-road-crash-analytics, folder `src/`). They run in the order `load_data.py` → `features.py` (imported) → `model.py`. The listings below are included directly from those files, so they are exactly the code that produced the results.

B.1 Model-building code, explained

The core of the modelling is a short block in `model.py`. It is shown first with an explanation; the complete files follow in Section B.2.

Predictors and target. The 16 categorical and 7 numeric predictors are listed by name. *Street lighting* is removed for the main run because of leakage. Missing categories become an explicit `"Missing"` level, and crashes are split by year.

Listing 1: Predictor lists, leakage exclusion, target and temporal split (`src/model.py`).

```
CAT = ["Urbanisation", "Speed limit", "Weather", "Natural lighting", "Surface condition", "Road surface",
       "Alignment", "Type of location", "Road classification (admin)", "First impact type", "Key TU type",
       "Two-hour intervals", "Day of week of crash", "Street lighting", "School zone active",
       "Signals operation", "Other traffic control"]
NUM = ["No. of traffic units involved", "has_pedestrian", "has_motorcycle", "has_pedal_cycle",
       "has_heavy_vehicle", "hit_tree_or_pole", "has_parked_vehicle"]

import sys
# Street lighting 'Unknown / not stated' is recorded for 63% of tow-away crashes but 0.6% of fatal ones:
# its completeness depends on the outcome (recording artefact / leakage), so the main model excludes it.
LEAKY = ["Street lighting"]
SPEC = sys.argv[1] if len(sys.argv) > 1 else "main"
if SPEC == "main":
    CAT = [f for f in CAT if f not in LEAKY]
SUFFIX = "" if SPEC == "main" else "_with_street_lighting"

c, _ = build()
X, y = c[CAT + NUM].copy(), c["ksi"]
X[CAT] = X[CAT].fillna("Missing").astype(str)
train, test = c["Year of crash"] <= 2023, c["Year of crash"] == 2024
Xtr, Xte, ytr, yte = X[train], X[test], y[train], y[test]
print("SPEC", SPEC); print(f"train {len(Xtr):,} (KSI {ytr.mean():.1%}) | test {len(Xte):,} (KSI {yte.mean():.1%})")
```

The three models. Each model is a scikit-learn `Pipeline`, so the encoding is learned from training data only. Logistic regression uses the one-hot encoder `ohe`. Gradient boosting uses an ordinal encoder and is told which columns are categorical. Both use balanced class weights.

Listing 2: Encoders, model definitions and hyperparameters (`src/model.py`).

```
ohe = ColumnTransformer([("cat", OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=50), CAT)],
                        remainder="passthrough")
models = {
    "Baseline (always 'not KSI')": Pipeline([("m", DummyClassifier(strategy="prior"))]),
    "Logistic regression": Pipeline([("prep", ohe), ("m", LogisticRegression(max_iter=3000, C=0.5,
                                                                             class_weight="balanced"))]),
    "Gradient boosting": Pipeline([
        ("prep", ColumnTransformer([("cat", OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1,
                                                           encoded_missing_value=-1), CAT)],
                                   remainder="passthrough")),
        ("m", HistGradientBoostingClassifier(categorical_features=list(range(len(CAT))), learning_rate=0.05,
                                             max_iter=400, max_leaf_nodes=31, l2_regularization=1.0,
```

```
                                            class_weight="balanced", random_state=42))]),
}
```

Fitting and scoring. Each pipeline is fitted on 2020–2023 and scored on 2024, and the metrics in Table 10 are computed at the 0.5 threshold.

Listing 3: Fitting, prediction and evaluation metrics (`src/model.py`).

```
results, probs = [], {}
for name, pipe in models.items():
    pipe.fit(Xtr, ytr)
    p = pipe.predict_proba(Xte)[:, 1]
    probs[name] = p
    pred = (p >= 0.5).astype(int)
    results.append({"model": name, "ROC-AUC": roc_auc_score(yte, p), "PR-AUC": average_precision_score(yte, p),
                    "recall_KSI": recall_score(yte, pred, zero_division=0),
                    "precision_KSI": precision_score(yte, pred, zero_division=0),
                    "F1_KSI": f1_score(yte, pred, zero_division=0),
                    "balanced_acc": balanced_accuracy_score(yte, pred),
                    "fatal_crashes_flagged": float(pred[c.loc[test, "fatal"].values == 1].mean())})
res = pd.DataFrame(results).set_index("model").round(3)
print(res.to_string())
res.to_csv(ROOT / "reports" / f"model_results{SUFFIX}.csv")
```

Worked examples. The two example crashes are selected, and each predictor's logistic regression contribution and gradient boosting's staged probabilities are extracted (Tables 12 and 13).

Listing 4: Worked-example extraction (`src/model.py`).

```
# Worked examples for the appendix: how each model scores one high-risk and one low-risk 2024 crash
test_rows = c.loc[test].assign(p_lr=probs["Logistic regression"], p_gb=probs["Gradient boosting"])
quiet = ["has_pedestrian", "has_pedal_cycle", "has_heavy_vehicle", "has_parked_vehicle"]
profiles = {
    "high_risk": (test_rows["has_motorcycle"].eq(1) & test_rows["Speed limit"].eq("100 km/h")
                  & test_rows["First impact type"].eq("Vehicle - Object") & test_rows["ksi"].eq(1)),
    "low_risk": (test_rows["Key TU type"].eq("Car (sedan/hatch)") & test_rows["Speed limit"].eq("60 km/h")
                 & test_rows["First impact type"].eq("Rear end") & test_rows["ksi"].eq(0)
                 & test_rows[quiet + ["has_motorcycle", "hit_tree_or_pole"]].sum(axis=1).eq(0)),
}
lr_prep, lr_m = lr_fit.named_steps["prep"], lr_fit.named_steps["m"]
gb_prep = models["Gradient boosting"].named_steps["prep"]
enc_names = [n.replace("cat__", "").replace("remainder__", "") for n in lr_prep.get_feature_names_out()]
examples = {}
for label, mask in profiles.items():
    grp = test_rows[mask]
    row = grp.iloc[(grp.p_lr - grp.p_lr.median()).abs().argsort().iloc[0]]  # typical member of the group
    x_raw = X.loc[[row.name]]
    x_enc = lr_prep.transform(x_raw)
    x_enc = x_enc.toarray()[0] if hasattr(x_enc, "toarray") else np.asarray(x_enc)[0]
    contrib = {enc_names[j]: float(lr_m.coef_[0][j] * x_enc[j]) for j in np.flatnonzero(x_enc)}
    logit = float(lr_m.intercept_[0] + sum(contrib.values()))
    staged = [float(pp[0, 1]) for pp in gb_fit.staged_predict_proba(gb_prep.transform(x_raw))]
    examples[label] = {
        "crash_id": int(row["Crash ID"]), "actual_ksi": int(row["ksi"]),
        "degree": row["Degree of crash - detailed"],
        "predictors": {k: (v.item() if hasattr(v, "item") else v) for k, v in x_raw.iloc[0].items()},
        "lr_intercept": float(lr_m.intercept_[0]), "lr_contributions": contrib,
        "lr_logit": logit, "lr_probability": float(1 / (1 + np.exp(-logit))),
        "gb_staged_probability": {str(m): staged[m - 1] for m in [1, 5, 10, 25, 50, len(staged)]},
        "gb_probability": float(row["p_gb"]),
    }
json.dump(examples, open(ROOT / "reports" / f"worked_examples{SUFFIX}.json", "w"), indent=2)
print(json.dumps(examples, indent=1))
```

B.2 Complete source files

Listing 5: `src/features.py`: cleaning, target definition (line 23) and TRAFFIC UNIT flags (lines 33–46).

```
"""Shared cleaning and feature engineering for the NSW crash data."""
import numpy as np
import pandas as pd
from load_data import load

HOUR_ORDER = ["00:01 - 01:59", "02:00 - 03:59", "04:00 - 05:59", "06:00 - 07:59",
              "08:00 - 09:59", "10:00 - 11:59", "12:00 - 13:59", "14:00 - 15:59",
              "16:00 - 17:59", "18:00 - 19:59", "20:00 - 21:59", "22:00 - Midnight"]
DAY_ORDER = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
SEVERITY_ORDER = ["Fatal", "Serious Injury", "Moderate Injury", "Minor/Other Injury",
                  "Non-casualty (towaway)"]
REGION_MAP = {"Syd-Newc-Woll Gtr conurbation": "Greater Sydney/Newcastle/Wollongong",
              "Rest of NSW - Urban": "Regional urban",
              "Rest of NSW - Rural": "Regional rural"}
NSW_BBOX = dict(lat=(-37.6, -28.0), lon=(140.9, 159.2))  # 159 = Lord Howe Island


def clean_crash(c):
    c = c.copy()
    # Crash year inside 2020-2024 only (2019 crashes were reported late, in 2020)
    c = c[c["Year of crash"].between(2020, 2024)]
    # Target: killed or seriously injured (KSI)
    c["ksi"] = c["Degree of crash - detailed"].isin(["Fatal", "Serious Injury"]).astype(int)
    c["fatal"] = (c["Degree of crash"] == "Fatal").astype(int)
    c["region"] = c["Conurbation 1"].map(REGION_MAP)  # 'Unknown' -> NaN
    c["speed_kmh"] = pd.to_numeric(c["Speed limit"].str.replace(" km/h", ""), errors="coerce")
    # Coordinates outside NSW are treated as missing
    bad = ~(c.Latitude.between(*NSW_BBOX["lat"]) & c.Longitude.between(*NSW_BBOX["lon"]))
    c.loc[bad, ["Latitude", "Longitude"]] = np.nan
    return c


def tu_flags(t):
    """Crash-level flags describing who/what was involved (from the TRAFFIC UNIT table)."""
    g = t["TU type group"]
    flags = pd.DataFrame({
        "Crash ID": t["Crash ID"],
        "has_pedestrian": g.eq("Pedestrian"),
        "has_motorcycle": g.eq("Motorcycle"),
        "has_pedal_cycle": g.eq("Pedal cycle"),
        "has_heavy_vehicle": g.isin(["Heavy rigid truck", "Articulated truck", "Bus"]),
        "hit_tree_or_pole": t["Object hit 1"].isin(["Tree/bush", "Utility pole"])
                            | t["Object hit 2"].isin(["Tree/bush", "Utility pole"]),
        "has_parked_vehicle": t["TU controlled flag"].eq("N"),
    })
    return flags.groupby("Crash ID").max().astype(int).reset_index()


def build():
    c = clean_crash(load("crash"))
    t = load("traffic_unit")
    return c.merge(tu_flags(t), on="Crash ID", how="left"), t
```

Listing 6: `src/model.py`: complete modelling script.

```
"""Baseline severity models: predict whether a crash is fatal or serious (KSI).

Temporal split: train on 2020-2023, test on 2024 (mimics using past data to flag future risk).
Features only describe the crash circumstances, never the injury counts (which define the target).
"""
from pathlib import Path
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.dummy import DummyClassifier
from sklearn.inspection import permutation_importance
from sklearn.metrics import (roc_auc_score, average_precision_score, roc_curve, precision_recall_curve,
                             recall_score, precision_score, f1_score, balanced_accuracy_score,
    brier_score_loss)
from features import build

ROOT = Path(__file__).resolve().parents[1]
FIG = ROOT / "reports" / "figures"
BLUE, ORANGE, AQUA, INK2, GRID, SURFACE = "#2a78d6", "#eb6834", "#1baf7a", "#52514e", "#e4e3df", "#
    fcfcfb"
plt.rcParams.update({"figure.facecolor": SURFACE, "axes.facecolor": SURFACE, "savefig.facecolor":
    SURFACE,
                     "axes.edgecolor": GRID, "axes.grid": True, "grid.color": GRID, "axes.spines.top":
    False,
                     "axes.spines.right": False, "axes.titleweight": "bold", "axes.titlelocation": "left
    ",
                     "xtick.color": INK2, "ytick.color": INK2, "axes.labelcolor": INK2})

CAT = ["Urbanisation", "Speed limit", "Weather", "Natural lighting", "Surface condition", "Road surface"
    ,
       "Alignment", "Type of location", "Road classification (admin)", "First impact type", "Key TU type
    ",
       "Two-hour intervals", "Day of week of crash", "Street lighting", "School zone active",
       "Signals operation", "Other traffic control"]
NUM = ["No. of traffic units involved", "has_pedestrian", "has_motorcycle", "has_pedal_cycle",
       "has_heavy_vehicle", "hit_tree_or_pole", "has_parked_vehicle"]

import sys
# Street lighting 'Unknown / not stated' is recorded for 63% of tow-away crashes but 0.6% of fatal ones:
# its completeness depends on the outcome (recording artefact / leakage), so the main model excludes it.
LEAKY = ["Street lighting"]
SPEC = sys.argv[1] if len(sys.argv) > 1 else "main"
if SPEC == "main":
    CAT = [f for f in CAT if f not in LEAKY]
SUFFIX = "" if SPEC == "main" else "_with_street_lighting"

c, _ = build()
X, y = c[CAT + NUM].copy(), c["ksi"]
X[CAT] = X[CAT].fillna("Missing").astype(str)
train, test = c["Year of crash"] <= 2023, c["Year of crash"] == 2024
Xtr, Xte, ytr, yte = X[train], X[test], y[train], y[test]
print("SPEC", SPEC); print(f"train {len(Xtr):,} (KSI {ytr.mean():.1%}) | test {len(Xte):,} (KSI {yte.
    mean():.1%})")

ohe = ColumnTransformer([("cat", OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=50),
    CAT)],
                        remainder="passthrough")
models = {
    "Baseline (always 'not KSI')": Pipeline([("m", DummyClassifier(strategy="prior"))]),
    "Logistic regression": Pipeline([("prep", ohe), ("m", LogisticRegression(max_iter=3000, C=0.5,
                                                                             class_weight="balanced"))
    ]),
    "Gradient boosting": Pipeline([
        ("prep", ColumnTransformer([("cat", OrdinalEncoder(handle_unknown="use_encoded_value",
    unknown_value=-1,
                                                           encoded_missing_value=-1), CAT)],
                                   remainder="passthrough")),
        ("m", HistGradientBoostingClassifier(categorical_features=list(range(len(CAT))), learning_rate
    =0.05,
                                             max_iter=400, max_leaf_nodes=31, l2_regularization=1.0,
                                             class_weight="balanced", random_state=42))]),
}

results, probs = [], {}
for name, pipe in models.items():
    pipe.fit(Xtr, ytr)
    p = pipe.predict_proba(Xte)[:, 1]
```

```
    probs[name] = p
    pred = (p >= 0.5).astype(int)
    results.append({"model": name, "ROC-AUC": roc_auc_score(yte, p), "PR-AUC": average_precision_score(yte, p),
                    "recall_KSI": recall_score(yte, pred, zero_division=0),
                    "precision_KSI": precision_score(yte, pred, zero_division=0),
                    "F1_KSI": f1_score(yte, pred, zero_division=0),
                    "balanced_acc": balanced_accuracy_score(yte, pred),
                    "fatal_crashes_flagged": float(pred[c.loc[test, "fatal"].values == 1].mean())})
res = pd.DataFrame(results).set_index("model").round(3)
print(res.to_string())
res.to_csv(ROOT / "reports" / f"model_results{SUFFIX}.csv")

# Diagnostics: training-set fit (overfitting check), model internals, and year-by-year stability
from sklearn.base import clone
diag = {"n_train": int(len(Xtr)), "n_test": int(len(Xte)),
        "ksi_rate_train": float(ytr.mean()), "ksi_rate_test": float(yte.mean())}
for name in ["Logistic regression", "Gradient boosting"]:
    p_tr = models[name].predict_proba(Xtr)[:, 1]
    diag[name] = {"train_roc_auc": roc_auc_score(ytr, p_tr), "test_roc_auc": roc_auc_score(yte, probs[name]),
                  "train_pr_auc": average_precision_score(ytr, p_tr), "test_pr_auc": average_precision_score(yte, probs[name])}
lr_fit = models["Logistic regression"]
diag["Logistic regression"]["n_encoded_columns"] = int(len(lr_fit.named_steps["prep"].get_feature_names_out()))
diag["Logistic regression"]["solver_iterations"] = int(lr_fit.named_steps["m"].n_iter_[0])
gb_fit = models["Gradient boosting"].named_steps["m"]
diag["Gradient boosting"]["boosting_iterations_used"] = int(gb_fit.n_iter_)
diag["Gradient boosting"]["early_stopping_active"] = bool(gb_fit.do_early_stopping_)
# Leave-one-year-out within the training years (2020-2023): train on 3 years, score the held-out year
years = c.loc[train, "Year of crash"]
loyo = {}
for name in ["Logistic regression", "Gradient boosting"]:
    loyo[name] = {}
    for yr in sorted(years.unique()):
        fit = clone(models[name]).fit(Xtr[years != yr], ytr[years != yr])
        loyo[name][int(yr)] = roc_auc_score(ytr[years == yr], fit.predict_proba(Xtr[years == yr])[:, 1])
diag["leave_one_year_out_roc_auc"] = loyo
json.dump(diag, open(ROOT / "reports" / f"model_diagnostics{SUFFIX}.json", "w"), indent=2)
print(json.dumps(diag, indent=1))

# ROC and PR curves
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for (name, p), color in zip([(k, v) for k, v in probs.items() if "Baseline" not in k], [BLUE, ORANGE]):
    fpr, tpr, _ = roc_curve(yte, p)
    axes[0].plot(fpr, tpr, lw=2, color=color, label=f"{name} (AUC {roc_auc_score(yte, p):.3f})")
    pr, rc, _ = precision_recall_curve(yte, p)
    axes[1].plot(rc, pr, lw=2, color=color, label=f"{name} (AP {average_precision_score(yte, p):.3f})")
axes[0].plot([0, 1], [0, 1], ls="--", color=INK2, lw=1, label="Random")
axes[1].axhline(yte.mean(), ls="--", color=INK2, lw=1, label=f"Random ({yte.mean():.1%} KSI)")
axes[0].set(title="ROC curve, 2024 test set", xlabel="False positive rate", ylabel="True positive rate")
axes[1].set(title="Precision-recall curve, 2024 test set", xlabel="Recall", ylabel="Precision", ylim=(0, 1))
for ax in axes:
    ax.legend(frameon=False, fontsize=8)
fig.tight_layout(); fig.savefig(FIG / f"07_model_roc_pr{SUFFIX}.png", dpi=200); plt.close(fig)

# Logistic regression odds ratios (top +/-)
lr = models["Logistic regression"]
names = lr.named_steps["prep"].get_feature_names_out()
coef = pd.Series(lr.named_steps["m"].coef_[0], index=names)
coef.index = coef.index.str.replace("cat__", "").str.replace("remainder__", "")
odds = np.exp(coef).sort_values()
odds.to_csv(ROOT / "reports" / f"logreg_odds_ratios{SUFFIX}.csv", header=["odds_ratio"])
top = pd.concat([odds.head(10), odds.tail(12)])
fig, ax = plt.subplots(figsize=(8, 7))
ax.barh(top.index, top.values - 1, left=1, color=[ORANGE if v > 1 else BLUE for v in top.values], height=0.6)
ax.axvline(1, color=INK2, lw=1); ax.set_xscale("log"); ax.grid(axis="y", visible=False)
ticks = [0.2, 0.3, 0.5, 1, 2, 3, 5, 8]
ax.set_xticks(ticks, [str(t) for t in ticks]); ax.minorticks_off()
ax.set_title("Logistic regression: strongest odds ratios for KSI")
```

```
ax.set_xlabel("Odds ratio (log scale; >1 raises the odds of a fatal/serious outcome)")
fig.tight_layout(); fig.savefig(FIG / f"08_logreg_odds_ratios{SUFFIX}.png", dpi=200); plt.close(fig)

# Permutation importance (gradient boosting, on the test set, metric = ROC-AUC)
gb = models["Gradient boosting"]
pi = permutation_importance(gb, Xte, yte, scoring="roc_auc", n_repeats=5, random_state=42, n_jobs=-1)
imp = pd.Series(pi.importances_mean, index=CAT + NUM).sort_values()
imp.to_csv(ROOT / "reports" / f"gb_permutation_importance{SUFFIX}.csv", header=["auc_drop"])
fig, ax = plt.subplots(figsize=(8, 6.5))
ax.barh(imp.index, imp.values, color=BLUE, height=0.6); ax.grid(axis="y", visible=False)
ax.set_title("Gradient boosting: permutation importance")
ax.set_xlabel("Drop in ROC-AUC when the feature is shuffled")
fig.tight_layout(); fig.savefig(FIG / f"09_gb_permutation_importance{SUFFIX}.png", dpi=200); plt.close(
    fig)

# Calibration / risk deciles for gradient boosting
p = probs["Gradient boosting"]
dec = pd.DataFrame({"p": p, "y": yte.values}).assign(decile=lambda d: pd.qcut(d.p, 10, labels=False) +
    1)
cal = dec.groupby("decile").agg(mean_pred=("p", "mean"), actual_ksi=("y", "mean"), n=("y", "size"))
print("\nRisk deciles (GB):\n", cal.round(3).to_string())
print("\nTop permutation importances:\n", imp.sort_values(ascending=False).head(10).round(4).to_string()
    )
print("\nTop odds ratios:\n", odds.tail(12).round(2).to_string(), "\n", odds.head(8).round(2).to_string
    ())
json.dump({"n_train": int(len(Xtr)), "n_test": int(len(Xte)), "ksi_rate_test": float(yte.mean()),
           "top_decile_ksi": float(cal.actual_ksi.iloc[-1]), "bottom_decile_ksi": float(cal.actual_ksi.
    iloc[0])},
          open(ROOT / "reports" / f"model_summary{SUFFIX}.json", "w"), indent=2)

# Worked examples for the appendix: how each model scores one high-risk and one low-risk 2024 crash
test_rows = c.loc[test].assign(p_lr=probs["Logistic regression"], p_gb=probs["Gradient boosting"])
quiet = ["has_pedestrian", "has_pedal_cycle", "has_heavy_vehicle", "has_parked_vehicle"]
profiles = {
    "high_risk": (test_rows["has_motorcycle"].eq(1) & test_rows["Speed limit"].eq("100 km/h")
                  & test_rows["First impact type"].eq("Vehicle - Object") & test_rows["ksi"].eq(1)),
    "low_risk": (test_rows["Key TU type"].eq("Car (sedan/hatch)") & test_rows["Speed limit"].eq("60 km/h
    ")
                 & test_rows["First impact type"].eq("Rear end") & test_rows["ksi"].eq(0)
                 & test_rows[quiet + ["has_motorcycle", "hit_tree_or_pole"]].sum(axis=1).eq(0)),
}
lr_prep, lr_m = lr_fit.named_steps["prep"], lr_fit.named_steps["m"]
gb_prep = models["Gradient boosting"].named_steps["prep"]
enc_names = [n.replace("cat__", "").replace("remainder__", "") for n in lr_prep.get_feature_names_out()]
examples = {}
for label, mask in profiles.items():
    grp = test_rows[mask]
    row = grp.iloc[(grp.p_lr - grp.p_lr.median()).abs().argsort().iloc[0]]  # typical member of the
    group
    x_raw = X.loc[[row.name]]
    x_enc = lr_prep.transform(x_raw)
    x_enc = x_enc.toarray()[0] if hasattr(x_enc, "toarray") else np.asarray(x_enc)[0]
    contrib = {enc_names[j]: float(lr_m.coef_[0][j] * x_enc[j]) for j in np.flatnonzero(x_enc)}
    logit = float(lr_m.intercept_[0] + sum(contrib.values()))
    staged = [float(pp[0, 1]) for pp in gb_fit.staged_predict_proba(gb_prep.transform(x_raw))]
    examples[label] = {
        "crash_id": int(row["Crash ID"]), "actual_ksi": int(row["ksi"]),
        "degree": row["Degree of crash - detailed"],
        "predictors": {k: (v.item() if hasattr(v, "item") else v) for k, v in x_raw.iloc[0].items()},
        "lr_intercept": float(lr_m.intercept_[0]), "lr_contributions": contrib,
        "lr_logit": logit, "lr_probability": float(1 / (1 + np.exp(-logit))),
        "gb_staged_probability": {str(m): staged[m - 1] for m in [1, 5, 10, 25, 50, len(staged)]},
        "gb_probability": float(row["p_gb"]),
    }
json.dump(examples, open(ROOT / "reports" / f"worked_examples{SUFFIX}.json", "w"), indent=2)
print(json.dumps(examples, indent=1))
```

Listing 7: `src/load_data.py`: conversion of the TfNSW workbooks to CSV.

```
"""Convert the raw TfNSW crash workbooks to CSV for faster loading."""
from pathlib import Path
import pandas as pd

```

```
RAW = Path(__file__).resolve().parents[1] / "data" / "raw"
OUT = Path(__file__).resolve().parents[1] / "data" / "processed"

FILES = {
    "crash": "nsw_road_crash_data_2020-2024_crash.xlsx",
    "traffic_unit": "nsw_road_crash_data_2020-2024_traffic_unit.xlsx",
}


def convert():
    OUT.mkdir(parents=True, exist_ok=True)
    for name, fname in FILES.items():
        df = pd.read_excel(RAW / fname, engine="openpyxl")
        df.to_csv(OUT / f"{name}.csv", index=False)
        print(name, df.shape)


def load(name):
    return pd.read_csv(OUT / f"{name}.csv", low_memory=False)


if __name__ == "__main__":
    convert()
```